# Übungsblatt 2: RISC-V Basics & Venus Onboarding

**Mikrocomputertechnik 1**

## Zielsetzung

In diesem Übungsblatt lernen Sie die Philosophie der RISC-V-Architektur kennen und schreiben Ihre ersten eigenen Programme. Sie machen sich mit den ABI-Registernamen vertraut, verstehen den Unterschied zwischen R-Type- und I-Type-Befehlen und nutzen den Venus-Simulator für schrittweises Debugging (Tracing).

## Aufgabe 1: Architektur und ABI

Bevor wir Code schreiben, müssen die Spielregeln der Hardware sitzen. Beantworten Sie die folgenden Fragen anhand der Vorlesung.

### Load/Store-Architektur

RISC-V ist eine strikte Load/Store-Architektur. Was bedeutet das für Berechnungen in der ALU in Bezug auf den Arbeitsspeicher (RAM)?

### Die ABI (Application Binary Interface)

Warum sollten Sie im Assembler-Code ABI-Namen (wie `t0`, `a0`) anstelle der reinen Hardware-Namen (wie `x5`, `x10`) nutzen?

### Das Zero-Register

Was passiert physikalisch, wenn Sie versuchen, das Ergebnis einer Addition in das Register `x0` (Zero) zu speichern?

### Ihre Lösung

*(Notieren Sie Ihre Antworten hier.)*

## Aufgabe 2: Immediates und die 12-Bit-Grenze

Wir wollen Konstanten in unseren Code einbauen (I-Type-Befehle).

### Konstanten-Limit

Sie möchten den Wert 2048 in das Register `t0` laden. Warum schlägt der Befehl `addi t0, x0, 2048` fehl? Welche Alternative (Pseudo-Instruktion) sollten Sie stattdessen nutzen?

### Fehlende Befehle

In der RISC-V-Basisarchitektur (RV32I) gibt es `sub` (Subtraktion zweier Register), aber keinen Befehl `subi` (Subtraktion einer Konstante). Warum haben die Chip-Designer diesen Befehl weggelassen?

### Ihre Lösung

*(Notieren Sie Ihre Antworten hier.)*

## Aufgabe 3: Venus Onboarding — Die erste Gleichung

Öffnen Sie den Venus-Simulator. Setzen Sie die Mathematik-Aufgabe aus der Vorlesung um.

Gegeben ist die Gleichung

$$Y = (A + B) - (C + D)$$

mit den Startwerten $A = 5$, $B = 10$, $C = 3$, $D = 2$.

### Aufgabenstellung

1. Nutzen Sie das `.text`-Segment und das Label `main:` als Einstiegspunkt.
2. Laden Sie die vier Konstanten über Pseudo-Instruktionen (`li`) in die temporären Register `t0` bis `t3`.
3. Berechnen Sie die beiden Klammern (Zwischensummen) und speichern Sie diese in `t4` und `t5`.
4. Subtrahieren Sie die Zwischensummen und speichern Sie das finale Ergebnis ($Y$) in `t6`.
5. Simulations-Check: Klicken Sie sich mit dem Step-Button durch den Code. Verifizieren Sie nach dem letzten Schritt, ob in `t6` der Wert 10 steht.
6. Konsolenausgabe (Venus-ABI): Nutzen Sie `ecall`, um das Ergebnis auszugeben. Laden Sie den Dienstcode 1 in `a0` und den Wert aus `t6` in `a1`.

### Ihre Lösung

*(Fügen Sie Ihren Assembler-Code hier ein.)*

## Aufgabe 4: Bit-Shifting und der x0-Trick

Die ALU der Basisarchitektur (RV32I) besitzt keine Hardware-Einheit für Multiplikation. Schiebeoperationen (Shifts) multiplizieren jedoch schnell mit Potenzen von 2.

### Aufgabenstellung in Venus

1. Laden Sie die Zahl 5 in das Register `t1`.
2. Multiplizieren Sie diese Zahl mit 8, indem Sie einen einzigen `slli`-Befehl (Shift Left Logical Immediate) anwenden und das Ergebnis in `t2` speichern. (Tipp: $2^3 = 8$ — um wie viele Bitstellen schieben?)
3. Negieren Sie das Ergebnis in `t2` mathematisch (aus $+40$ wird $-40$) und speichern Sie es in `t3`. Nutzen Sie dafür `sub` und den Trick mit dem Zero-Register (`x0`).

### Ihre Lösung

*(Fügen Sie Ihren Assembler-Code hier ein.)*

## Aufgabe 5: Maschinencode von Hand

In der Vorlesung haben Sie das R-Type-Format (opcode, rd, funct3, rs1, rs2, funct7) kennengelernt. Für die Kodierung nutzen wir bewusst die Hardware-Namen `x…`, weil sie den 5-Bit-Feldern entsprechen.

### Assemblieren

Assemblieren Sie den Befehl `add x9, x20, x21` von Hand zu einem 32-Bit-Wort (Hexadezimal). Nutzen Sie: opcode `0110011`, funct3 `000`, funct7 `0000000`.

### Decodieren

Decodieren Sie das Maschinenwort `0x007302B3`. Welcher Assembler-Befehl (mit ABI-Namen) steckt dahinter?

### Ihre Lösung

*(Notieren Sie Ihre Rechenwege hier.)*